

Projet en classe : Tappy

Dans les prochains cours (09 à 12), nous allons réaliser un jeu de type “endless runner” où le joueur contrôle un avion qui doit éviter les obstacles et collecter des étoiles pour marquer des points.

1.1 Cours 09 - Déplacer le décor

1.1.1 1. Révision des déplacements

1.1.1.1 Dans le script DéplacementDecor :

- Créer une variable de type `float` nommée `vitesse` et lui assigner une valeur de `1f`.
- Créer une variable de type `int` nommée `direction` et lui assigner une valeur de `1`.
- Créer une variable de type `float` nommée `limiteGauche` et lui assigner une valeur de `0f`.
- Créer une variable de type `float` nommée `limiteDroite` et lui assigner une valeur de `10f`.
- Dans la méthode `Update()`, ajouter le code déplacer le décor :
 - Utilise la fonction `Translate()` pour déplacer le décor horizontalement en fonction de la `vitesse` et de la `direction`.
 - Gardez la position `y` du décor constante : ex : `transform.Translate(vitesse * direction * Time.deltaTime, 0, 0);`
 - Si la `direction` est négative et que la position `x` du décor est inférieure ou égale à `limiteGauche`, remplacer le décor à la `limiteDroite`.
 - Si la `direction` est positive et que la position `x` du décor est supérieure ou égale à `limiteDroite`, remplacer le décor à la `limiteGauche`.

1.1.1.2 Dans l'inspecteur de Unity

- Pour le fond, mettre une `vitesse` de `1f`, une `direction` de `-1`, une `limiteGauche` de `-16.83` et une `limiteDroite` de `16.83`.
- Pour le bloc de nuages, mettre une `vitesse` de `2f`, une `direction` de `-1`, une `limiteGauche` de `-20.5` et une `limiteDroite` de `11.72`.

1.1.2 2. Ajouter les collisions

- Sur tous les éléments du décor, ajouter un composant `PolygonCollider2D`.
- Sur l'avion, ajouter un composant `BoxCollider2D`.
- Sur les étoiles, ajouter un composant `BoxCollider2D` et cocher la case `Is Trigger`.

1.1.3 3. Ajouter un rigidbody2d sur l'avion

- Mettez les paramètres du RigidBody2D de l'avion comme suit :
 - Mass : 1
 - Linear Damping : 1.2
 - Angular Damping : 1
 - Gravity Scale : 1
 - Constraints : Cocher Freeze Rotation

1.2 Cours 10 - Déplacer l'avion et gérer les collisions

1.2.1 4. Déplacer l'avion

1.2.1.1 Dans le script `DeplacementAvion` :

- Ajouter une variable publique de type `RigidBody2D` nommée `rigid`.
- Ajouter une variable publique de type `float` nommée `vitesse` et lui assigner une valeur de `0.2f`.
- Ajouter une variable de type `float` nommée `deplacementHorizontal`.
- Ajouter une variable de type `float` nommée `deplacementVertical`.
- Ajouter une variable de type `bool` nommée `estMort` et lui assigner une valeur de `false`.
- Dans la méthode `Start()`, assigner le composant `RigidBody2D` de l'avion à la variable `rigid` en utilisant `GetComponent<RigidBody2D>()`.
- Dans la méthode `Update()`, modifier la vitesse linéaire du `RigidBody2D` de l'avion :
`rigid.linearVelocity += new Vector2(deplacementHorizontal * vitesse, deplacementVertical * vitesse);`

1.2.2 5. Gérer les collisions avec le décor

1.2.2.1 Dans le script `DeplacementAvion` :

- Ajouter une méthode `OnCollisionEnter2D(Collision2D collision)`.
 - Dans cette méthode, si l'élément touché possède le tag "Decor" et que `estMort` est `false`, alors :
 - Assigner `true` à la variable `estMort`.
 - Retirer la contrainte de rotation du `RigidBody2D` de l'avion pour permettre à l'avion de tourner librement.
 - Appliquer une vitesse de rotation à l'avion pour simuler une chute en spirale. Par exemple : `rb.angularVelocity = 100f;`

1.2.3 6. Gérer les collisions avec les étoiles

1.2.3.1 Dans le script `DeplacementAvion` :

- Ajouter une méthode `OnTriggerEnter2D(Collider2D collision)`.
 - Dans cette méthode, si l'élément touché possède le tag "Etoile", alors :
 - Accéder au script `GestionEtoile` de l'étoile touchée en utilisant `collision.GetComponent<GestionEtoile>`
 - Appeler la méthode publique `Cacher` du script `GestionEtoile`

1.3 Cours 11 - Ajouter du son

1.3.1 7. Gestion du son

Sur l'objet de l'avion, ajouter un composant `AudioSource` et assigner le son de la musique et faire jouer la musique au démarrage du jeu et en boucle.

1.3.1.1 Dans le script `DeplacementAvion` :

- Créer deux variables de type `AudioClip` pour stocker les sons de la collision et de la collection d'étoiles et les assigner dans l'inspecteur.
- Enregistrer la référence du composant `AudioSource` de l'avion dans une variable.

1.3.1.2 Dans la fonction `OnCollisionEnter2D` du script `DeplacementAvion` :

- Ajouter une ligne de code pour jouer le son de la collision lorsque l'avion touche le décor. Par exemple : `audioSource.PlayOneShot(sonCollision);`

1.3.1.3 Dans la fonction `OnTriggerEnter2D` du script `DeplacementAvion` :

- Ajouter une ligne de code pour jouer le son de la collection d'étoiles lorsque l'avion touche une étoile. Par exemple : `audioSource.PlayOneShot(sonEtoile);`

1.4 Cours 12 - Afficher le pointage

1.4.1 8. Créer un canvas pour afficher le pointage

- Dans Unity, créer un objet `Canvas` et le nommer "CanvasPointage". Assurez-vous que le "Scale Mode" du `Canvas` est réglé sur "Scale With Screen Size" pour que le pointage s'adapte à différentes résolutions d'écran.
- Dans le `CanvasPointage`, créer un objet `TMPText` et le nommer "TextPointage".
- Positionner le `TextPointage` en haut à gauche de l'écran et ajuster sa taille et sa police pour qu'il soit bien visible.

1.4.1.1 Dans le script `DeplacementAvion` :

- Ajouter une variable publique de type `TMP_Text` nommée `textPointage` et l'assigner dans l'inspecteur en faisant glisser le `TextPointage` du `CanvasPointage`.

- Ajouter une variable de type `int` nommée `pointage` et lui assigner une valeur de 0.
- Dans la fonction `OnTriggerEnter2D`, après jouer le son de la collection d'étoiles, augmenter le pointage de 1 et mettre à jour le texte affiché dans `textPointage`.
- Utiliser l'interpolation de chaîne pour afficher le pointage : `textPointage.text = $"Pointage: {pointage}";`

1.5 Vidéo complet